

Отчёт по лабораторной работе №9

по дисциплине «Архитектура компьютера»

Дембирел Тимур Леонидович

Российский университет дружбы народов
Факультет физико-математических и естественных наук
Кафедра прикладной информатики и теории вероятностей
Москва 2025г.

Содержание

1	Цель работы	6
2	Выполнение лабораторной работы	7
3	Вывод	27

Список иллюстраций

2.1	Создаем	8
2.2	Вводим	9
2.3	Проверяем	9
2.4	Изменяем	10
2.5	Проверяем	10
2.6	Создаем	11
2.7	Запускаем	12
2.8	Устанавливаем и запускаем	12
2.9	Смотрим	13
2.10	Переключаемся	14
2.11	Включаем и устанавливаем	15
2.12	Смотрим	16
2.13	Выполняем и отслеживаем	16
2.14	Смотрим	17
2.15	Смотрим	17
2.16	Изменяем	17
2.17	Заменяем	17
2.18	Смотрим	18
2.19	Изменяем	19
2.20	Выходим	19
2.21	Копируем	20
2.22	Устанавливаем	20
2.23	Смотрим	21
2.24	Копируем	21
2.25	Меняем	22
2.26	Проверяем	22
2.27	Создаем и вводим	23
2.28	Проверяем	24
2.29	Запускаем и смотрим	24
2.30	Изменяем	25
2.31	Проверяем	26

Список таблиц

2.1	Основные команды, используемые при работе	7
-----	---	---

1 Цель работы

Приобретение навыков написания программ с использованием подпрограмм.
Знакомство с методами отладки при помощи GDB и его основными возможностями.

2 Выполнение лабораторной работы

В Таблица 2.1 приведены основные команды, используемые при работе с git, элементами программирования.

Таблица 2.1: Основные команды, используемые при работе

Команды	Описание
<code>touch</code>	Создание файла
<code>nasm</code>	Компилятор ассемблера NASM
<code>git add</code>	Добавление изменений в индекс
<code>git commit</code>	Создание коммита
<code>git push</code>	Отправка изменений на удаленный репозиторий
<code>ld</code>	Связывает объектные файлы в исполняемую программу
<code>mkdir</code>	Создает новую директорию
<code>GDB</code>	Производит отладку многих языков программирования
<code>quit</code>	Команда для выхода из отладчика
<code>disassemble</code>	Команда для просмотра дизассемблированного кода программы

Команды	Описание
„break“	Команда для создания точки останова
„info breakpoints“	Команда для вывода информации о всех установленных точках останова
„stepi(si)“	Позволяет выполнять программу по шагам
„info registers“	Команда для просмотра содержимого регистров

Создаем каталог для выполнения лабораторной работы № 9, переходим в него и создаем файл lab09-1.asm рис. 2.1.

```
timur@timur-VirtualBox:~$ mkdir ~/work/arch-pc/lab09
timur@timur-VirtualBox:~$ cd ~/work/arch-pc/lab09
timur@timur-VirtualBox:~/work/arch-pc/lab09$ touch lab09-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab09$
```

Рисунок 2.1: Создаем

Введем в файл lab09-1.asm текст программы из листинга 9.1 с помощью любого текстового редактора рис. 2.2

```

GNU nano 7.2                                lab09-1.asm
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB '2x+7=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
GLOBAL _start
_start:
;-----
; Основная программа
;-----
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax,x
call atoi

```

Рисунок 2.2: Вводим

Создаем исполняемый файл и проверяем его рис. 2.3.

```

timur@timur-VirtualBox:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
timur@timur-VirtualBox:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 12
2x+7=31
timur@timur-VirtualBox:~/work/arch-pc/lab09$

```

Рисунок 2.3: Проверяем

Изменяем текст программы, добавив подпрограмму `_subcalcul` в подпрограмму `_calcul` рис. 2.4.

```

GNU nano 7.2 lab09-1.asm
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB '2(3x-1)+7=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul
mov eax, result
call sprint
mov eax, [res]
call iprintLF
call quit
_subcalcul:
mov ebx, 3
mul ebx
sub eax, 1
ret

```

Рисунок 2.4: Изменяем

И также создаем исполняемый файл и проверяем его работу рис. 2.5.

```

timur@timur-VirtualBox:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
timur@timur-VirtualBox:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 5
2(3x-1)+7=35

```

Рисунок 2.5: Проверяем

Создаем файл lab09-2.asm с текстом программы из Листинга 9.2 рис. 2.6.

```
GNU nano 7.2 lab09-2.asm
SECTION .data
msg1: db "Hello, ",0x0
msg1Len: equ $ - msg1
msg2: db "world!",0xa
msg2Len: equ $ - msg2
SECTION .text
global _start
_start:
mov eax, 4
mov ebx, 1
mov ecx, msg1
mov edx, msg1Len
int 0x80
mov eax, 4
mov ebx, 1
mov ecx, msg2
mov edx, msg2Len
int 0x80
mov eax, 1
mov ebx, 0
int 0x80
```

Рисунок 2.6: Создаем

Получаем исходный файл с помощью отладчика GDB и запускаем команду рис. 2.7.

```

timur@timur-VirtualBox:~/work/arch-pc/lab09$ gdb lab09-2
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb) run
Starting program: /home/timur/work/arch-pc/lab09/lab09-2

This GDB supports auto-downloading debuginfo from the following URLs:
    <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0xf7ffc000
Hello, world!
[Inferior 1 (process 21601) exited normally]
(gdb)

```

Рисунок 2.7: Запускаем

Устанавливаем брейкпоинт на метку `_start`, с которой начинается выполнение любой ассемблерной программы, и запускаем её рис. 2.8.

```

(gdb) break _start
Breakpoint 4 at 0x8049000: file lab09-2.asm, line 9.
(gdb) run
Starting program: /home/timur/work/arch-pc/lab09/lab09-2

Breakpoint 4, _start () at lab09-2.asm:9
9      mov eax, 4
(gdb)

```

Рисунок 2.8: Устанавливаем и запускаем

Смотрим дисассимилированный код программы с помощью команды `disassemble` начиная с метки `_start` рис. 2.9.

```
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     $0x4,%eax
    0x08049005 <+5>:      mov     $0x1,%ebx
    0x0804900a <+10>:     mov     $0x804a000,%ecx
    0x0804900f <+15>:     mov     $0x8,%edx
    0x08049014 <+20>:     int     $0x80
    0x08049016 <+22>:     mov     $0x4,%eax
    0x0804901b <+27>:     mov     $0x1,%ebx
    0x08049020 <+32>:     mov     $0x804a008,%ecx
    0x08049025 <+37>:     mov     $0x7,%edx
    0x0804902a <+42>:     int     $0x80
    0x0804902c <+44>:     mov     $0x1,%eax
    0x08049031 <+49>:     mov     $0x0,%ebx
    0x08049036 <+54>:     int     $0x80
End of assembler dump.
(gdb) █
```

Рисунок 2.9: Смотрим

Переключимся на отображение команд с Intel'овским синтаксисом, введя команду `set disassembly-flavor intel` рис. 2.10.

```

(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     eax,0x4
    0x08049005 <+5>:      mov     ebx,0x1
    0x0804900a <+10>:     mov     ecx,0x804a000
    0x0804900f <+15>:     mov     edx,0x8
    0x08049014 <+20>:     int     0x80
    0x08049016 <+22>:     mov     eax,0x4
    0x0804901b <+27>:     mov     ebx,0x1
    0x08049020 <+32>:     mov     ecx,0x804a008
    0x08049025 <+37>:     mov     edx,0x7
    0x0804902a <+42>:     int     0x80
    0x0804902c <+44>:     mov     eax,0x1
    0x08049031 <+49>:     mov     ebx,0x0
    0x08049036 <+54>:     int     0x80
End of assembler dump.
(gdb) █

```

Рисунок 2.10: Переключаемся

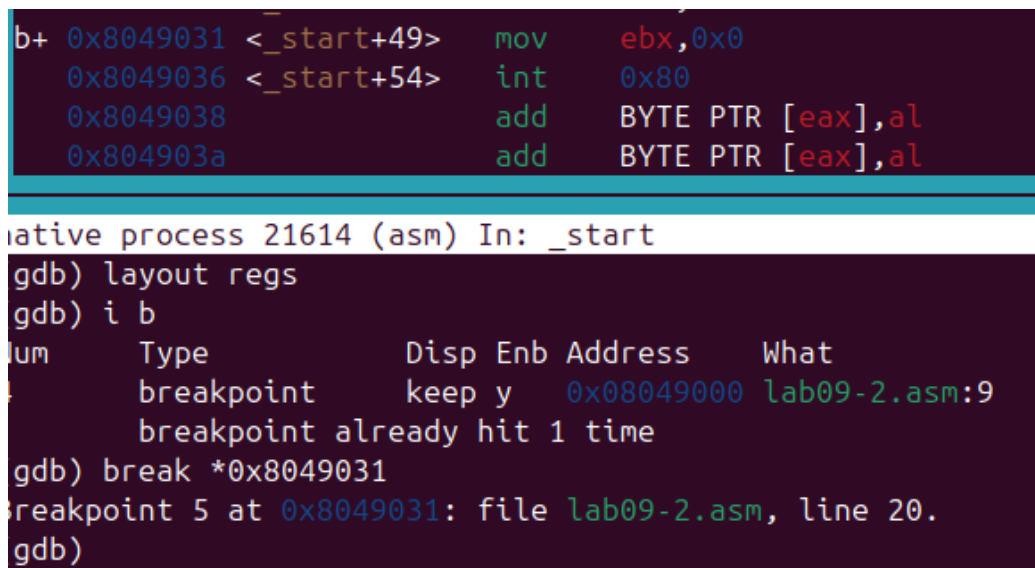
Различия отображения синтаксиса машинных команд в режимах АТТ и Intel:

1. Последовательность операндов: В синтаксисе АТТ операнды идут в обратном порядке — сначала указывается исходный операнд, затем принимающий. В синтаксисе Intel, как правило, сохраняется прямой порядок: принимающий операнд записывается первым, исходный — вторым.
2. Разделительные знаки: В синтаксисе АТТ для разделения операндов используются запятые. В синтаксисе Intel в качестве разделителей могут выступать как запятые, так и косые черты (/).
3. Указание размера операнда: В синтаксисе АТТ размер операнда задаётся пе-

ред ним с помощью префиксов: «b» (байт), «w» (слово), «l» (двойное слово), «q» (учетверённое слово). В синтаксисе Intel размер указывается после операнда через аналогичные суффиксы.

4. Обозначение непосредственных значений: В синтаксисе АТТ перед непосредственным значением (константой) ставится символ «\$». В синтаксисе Intel такой знак не используется.
5. Запись адресов памяти: В синтаксисе АТТ адреса заключаются в круглые скобки. В синтаксисе Intel адреса записываются без скобок.
6. Именованние регистров: В синтаксисе АТТ имена регистров предваряются символом «%». В синтаксисе Intel регистры могут обозначаться с префиксами «R» или «E» (например, RAX или EAX), но без дополнительных символов.

Включим режим псевдографики для более удобного анализа программы, проверим была ли установлена точка останова и установим точку останова предпоследней инструкции рис. 2.11.



```
b+ 0x8049031 <_start+49> mov    ebx,0x0
0x8049036 <_start+54> int     0x80
0x8049038          add     BYTE PTR [eax],al
0x804903a          add     BYTE PTR [eax],al

native process 21614 (asm) In: _start
gdb) layout regs
gdb) i b
um      Type           Disp Enb Address      What
      breakpoint      keep y  0x08049000 lab09-2.asm:9
      breakpoint already hit 1 time
gdb) break *0x8049031
breakpoint 5 at 0x8049031: file lab09-2.asm, line 20.
gdb)
```

Рисунок 2.11: Включаем и устанавливаем

Посмотрим информацию о всех установленных точках останова рис. 2.12.

```
(gdb) i b
Num      Type      Disp Enb Address      What
4        breakpoint keep y  0x08049000 lab09-2.asm:9
          breakpoint already hit 1 time
5        breakpoint keep y  0x08049031 lab09-2.asm:20
(gdb) █
```

Рисунок 2.12: Смотрим

Выполним 5 инструкций с помощью команды `stepi` (или `si`) и проследим за изменением значений регистров рис. 2.13.

```
timur@timur-VirtualBox: ~/work/arch-pc/lab09
Register group: general
eax      0x8      8      ecx      0x804a000    134520832
edx      0x8      8      ebx      0x1            1
esp      0xffffd020 0xffffd020  ebp      0x0            0x0
esi      0x0      0      edi      0x0            0
eip      0x8049016 0x8049016 <_start+22> eflags    0x202          [ IF ]
cs       0x23     35     ss       0x2b           43
ds       0x2b     43     es       0x2b           43
fs       0x0      0      gs       0x0            0

B+ 0x8049000 <_start>    mov     eax,0x4
0x8049005 <_start+5>    mov     ebx,0x1
0x804900a <_start+10>   mov     ecx,0x804a000
0x804900f <_start+15>   mov     edx,0x8
0x8049014 <_start+20>   int     0x80
>0x8049016 <_start+22>   mov     eax,0x4
0x804901b <_start+27>   mov     ebx,0x1
0x8049020 <_start+32>   mov     ecx,0x804a008
0x8049025 <_start+37>   mov     edx,0x7

native process 21614 (asm) In: _start L14 PC: 0x8049016
(gdb) i b
Num      Type      Disp Enb Address      What
4        breakpoint keep y  0x08049000 lab09-2.asm:9
          breakpoint already hit 1 time
5        breakpoint keep y  0x08049031 lab09-2.asm:20
(gdb) stepi
(gdb) stepi
(gdb) stepi
(gdb) stepi
(gdb) stepi
(gdb) █
```

Рисунок 2.13: Выполняем и отслеживаем

В результате выполнения команд менялись регистры: `ebx`, `ecx`, `edx`, `eax`, `eip`.

Посмотрим значение переменной `msg1` по имени рис. 2.14.

```
(gdb) x/1sb &msg1
0x804a000 <msg1>:      "Hello, "
(gdb)
```

Рисунок 2.14: Смотрим

Посмотрим значение переменной msg2 по адресу рис. 2.15.

```
(gdb) x/1sb 0x804a008
0x804a008 <msg2>:      "world!\n\034"
(gdb)
```

Рисунок 2.15: Смотрим

Изменим первый символ переменной msg1 рис. 2.16.

```
(gdb) set {char}&msg1='h'
(gdb) x/1sb &msg1
0x804a000 <msg1>:      "hello, "
(gdb)
```

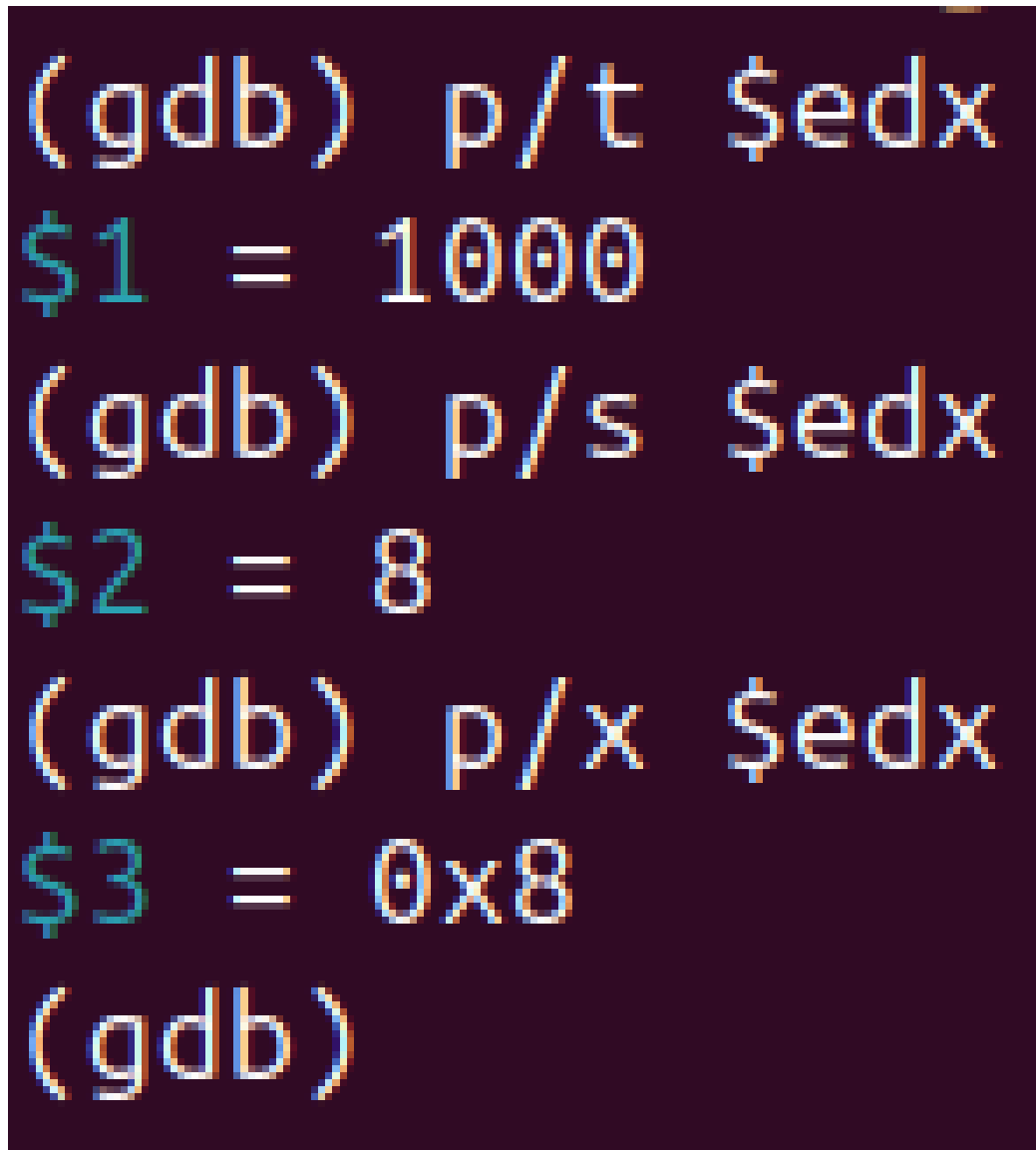
Рисунок 2.16: Изменяем

Заменяем любой символ во второй переменной msg2 рис. 2.17.

```
(gdb) set {char}&msg2='S'
(gdb) x/1sb &msg2
0x804a008 <msg2>:      "Sorld!\n\034"
(gdb)
```

Рисунок 2.17: Заменяем

Смотрим значение регистра `edx` в разных форматах рис. 2.18.



```
(gdb) p/t $edx
$1 = 1000

(gdb) p/s $edx
$2 = 8

(gdb) p/x $edx
$3 = 0x8

(gdb)
```

Рисунок 2.18: Смотрим

С помощью команды `set` изменим значение регистра `ebx` рис. 2.19.

```
(gdb) set $ebx='2'
(gdb) p/s $ebx
$4 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$5 = 2
(gdb) █
```

Рисунок 2.19: Изменяем

Команда без кавычек интерпретирует вводимое значение как числовую константу, а не как символьную строку.

Завершаем программу и выходим из GDB рис. 2.20.

```
Continuing.
Sorld!

Breakpoint 5, _start () at lab09-2.asm:20
(gdb) █
```

Рисунок 2.20: Выходим

Скопируем файл lab8-2.asm, созданный при выполнении лабораторной работы

№8, с программой выводящей на экран аргументы командной строки в файл с именем lab09-3.asm рис. 2.21.

```
timur@timur-VirtualBox:~/work/arch-pc/lab08$ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/lab09/lab09-3.asm
```

Рисунок 2.21: Копируем

Создаем исполняемый файл и запускаем его в отладчике GDB, указав аргументы ?@fig-022.

```
timur@timur-VirtualBox:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-3.lst lab09-3.asm
timur@timur-VirtualBox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-3 lab09-3.o
timur@timur-VirtualBox:~/work/arch-pc/lab09$ gdb --args lab09-3 1 2 '3'
```

{#fig-

022 width=90%

Установим точку останова перед первой инструкцией в программе и запустим ее рис. 2.22.

```
(gdb) b _start
Breakpoint 1 at 0x80490e8: file lab09-3.asm, line 5.
(gdb) run
Starting program: /home/timur/work/arch-pc/lab09/lab09-3 1 2 3

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0xf7ffc000

Breakpoint 1, _start () at lab09-3.asm:5
5      pop ecx ; Извлекаем из стека в `ecx` количество
(gdb) █
```

Рисунок 2.22: Устанавливаем

Смотрим позиции стека по разным адресам рис. 2.23.

```

(gdb) x/s *(void**)(esp + 4)
0xffffd1db:    "/home/timur/work/arch-pc/lab09/lab09-3"
(gdb) x/s *(void**)(esp + 8)
0xffffd202:    "1"
(gdb) x/s *(void**)(esp + 12)
0xffffd204:    "2"
(gdb) x/s *(void**)(esp + 16)
0xffffd206:    "3"
(gdb) x/s *(void**)(esp + 20)
0x0:    <error: Cannot access memory at address 0x0>
(gdb)

```

Рисунок 2.23: Смотрим

Шаг изменения адреса равен 4, потому что адресные регистры имеют размерность 32 бита(4 байта).

#Задание для самостоятельной работы

1. Преобразуйте программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции $f(x)$ как подпрограмму.

Копируем файл (из 1 задания из заданий для самостоятельной работы) tasklab8.asm в файл с именем lab09-3.asm рис. 2.24.

```

timur@timur-VirtualBox: ~/work/arch-pc/lab09$ cp ~/work/arch-pc/lab08/tasklab8.asm ~/work/arch-pc/lab09/lab09-3.asm

```

Рисунок 2.24: Копируем

Меняем текст программы, создавая подпрограмму рис. 2.25.

```

#include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB '10x-5=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax,x
call atoi
call _calcul
mov eax,result
call sprint
mov eax,[res]
call iprintLF
call quit
_calcul:
mov ebx,10
mul ebx
sub eax,5
mov [res],eax

```

Рисунок 2.25: Меняем

Создаем исполняемый файл и проверяем его рис. 2.26.

```

timur@timur-VirtualBox:~/work/arch-pc/lab09$ nasm -f elf lab09-4.asm
timur@timur-VirtualBox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-4 lab09-4.o
timur@timur-VirtualBox:~/work/arch-pc/lab09$ ./lab09-4
Введите x: 3
10x-5=25
timur@timur-VirtualBox:~/work/arch-pc/lab09$

```

Рисунок 2.26: Проверяем

2. В листинге 9.3 приведена программа вычисления выражения $(3 + 2) * 4 + 5$. При запуске данная программа дает неверный результат. Проверьте это. С

помощью отладчика GDB, анализируя изменения значений регистров, определите ошибку и исправьте ее.

Создаем файл для выполнения задания и вводим текст программы из листинга 9.3 рис. 2.27.

```
%include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения (3+2)*4+5
mov ebx,3
mov eax,2
add ebx,eax
mov ecx,4
mul ecx
add ebx,5
mov edi,ebx
; ---- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit
```

Рисунок 2.27: Создаем и вводим

Создаем исполняемый файл и проверяем его рис. 2.28.

```

timur@timur-VirtualBox:~/work/arch-pc/lab09$ nasm -f elf lab09-5.asm
timur@timur-VirtualBox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-5 lab09-5.o
timur@timur-VirtualBox:~/work/arch-pc/lab09$ ./lab09-5
Результат: 10
timur@timur-VirtualBox:~/work/arch-pc/lab09$ █

```

Рисунок 2.28: Проверяем

Результат неверный

Создаем исполняемый файл, запускаем его в отладчике GDB и смотрим на изменение регистров командой stepi(si) рис. 2.29.

```

--Register group: general--
eax      0x804a000      134520832      ecx      0x4          4
edx      0x0           0             ebx      0xa         10
esp      0xffffd020    0xffffd020    ebp      0x0         0x0
esi      0x0           0             edi      0xa         10
eip      0x8049105     0x8049105 <_start+29> eflags   0x10206        [ PF IF RF ]
cs       0x23          35            ss       0x2b         43
ds       0x2b          43            es       0x2b         43
fs       0x0           0             gs       0x0         0

0x80490f4 <_start+12> mov     ecx,0x4
0x80490f9 <_start+17> mul     ecx
0x80490fb <_start+19> add     ebx,0x5
0x80490fe <_start+22> mov     edi,ebx
0x8049100 <_start+24> mov     eax,0x04a000
>0x8049105 <_start+29> call    0x804900f <sprint>
0x804910a <_start+34> mov     eax,edi
0x804910c <_start+36> call    0x8049086 <iprintf>
0x8049111 <_start+41> call    0x80490db <quit>

native process 22945 (asm) In: _start L17 PC: 0x8049105
eax      0x804a000      134520832
ecx      0x4           4
edx      0x0           0
ebx      0xa           10
esp      0xffffd020    0xffffd020
ebp      0x0           0x0
esi      0x0           0
edi      0xa           10
eip      0x8049105     0x8049105 <_start+29>
eflags   0x10206        [ PF IF RF ]
--Type <RET> for more, q to quit, c to continue without paging-- █

```

Рисунок 2.29: Запускаем и смотрим

Изменяем текст программы для корректной работы рис. 2.30.

```
GNU nano 7.2
#include 'in_out.asm'
SECTION .data
msg: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
mov ebx,3
mov eax,2
add eax,ebx
mov ecx,4
mul ecx
add eax,5
mov edi,eax
mov eax,msg
call sprint
mov eax,edi
call iprintLF
call quit
```

Рисунок 2.30: Изменяем

Создаем исполняемый файл и проверяем его рис. 2.31.

```
timur@timur-VirtualBox:~/work/arch-pc/lab09$ nasm -f elf lab09-5.asm
timur@timur-VirtualBox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-5 lab09-5.o
timur@timur-VirtualBox:~/work/arch-pc/lab09$ ./lab09-5
Результат: 25
timur@timur-VirtualBox:~/work/arch-pc/lab09$
```

Рисунок 2.31: Проверяем

3 Вывод

В ходе данной лабораторной работы я приобрел навыки написания программ с использованием подпрограмм. Познакомился с методами отладки при помощи GDB и его основными возможностями.